Exercises: Relative Frequencies, Probabilities, and Conditional Probabilities

In the following, let us use the NLSY97 Monthly Employment and College Enrollment Transitions data file (available on Moodle): `nlsy97_transitions.parquet`.

I. Loading the data

1. Let us load the necessary packages:

```
# external (third-party) packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

import math
from pathlib import Path
```

2. Let us define the path (this time I use a `Path` object) to the project directory (I name it `ROOT` this time) and the path to the directory in which our data file is located (I name it `DATA` this time):

```
# Your path will almost certainly differ from the one below.
ROOT = Path(r'C:\Users\PC_DS_ECON_5\Desktop\data-analytics-python-wise2627')

# The relative path could also differ from the one below.
DATA = ROOT / 'local' / 'data'

print(ROOT)
print(DATA)
```

3. Let us read in the `nlsy97_transitions.parquet` file using `pandas.read_parquet()`:

```
df = pd.read_parquet(DATA / 'nlsy97_transitions.parquet')
```

(Reminder: To read the file, one needs a Parquet engine, such as the one provided by the `pyarrow` package. The package can be installed in the usual way with `pip install pyarrow` after activating the virtual environment.)

4. Let us keep only observations from the `'Cross-sectional'` subsample:

```
df = (
    df
        .loc[df['sample'].eq('Cross-sectional'),:]
        .reset_index(drop=True)
        .copy()
)
```

II. Relative frequencies and probabilities

In what follows, let us focus on a particular month.

Let us choose a month randomly.

To do so, let us first collect the set of unique values of month in our data frame using `drop_duplicates()`:

```
all_months = (
    df['year_month'].drop_duplicates().to_list().copy()
)
print(all_months)
type(all_months)
```

For carrying out the random draw, let us use (pseudo-) random number generator `numpy.random` module from `numpy` for this purpose.

Let us first create a particular random number generator object and let us call it `rng`.

```
seed = 7982456
rng = np.random.default_rng(seed)
type(rng)
```

By specifying a `seed` number, we can ensure that the same random draws occur at every execution of the entire code. This is good for reproducibility. But of course, it makes it possible to obtain a desired result via the choice of the seed value.

For the random draw, let us use the method `choice`:

```
chosen_month = rng.choice(
        a=all_months
    )
print(chosen_month)
```

With the current seed `7982456`, the chosen month is January 2015.

Let us restrict our sample to observations from the chosen month:

```
df1 = (
    df
        .loc[df['year_month'].eq(chosen_month),:]
        .reset_index(drop=True)
        .copy()
)
```

(a) How would you check that the value of emp_category is never missing in our selected dataset?

```
# your code goes here
```

Let us create a variable called `employed` that is equal to `1` if the individual is employed (that is, `emp_category=='employed'`) and `0` otherwise. For this purpose, let us use the function `where` from `numpy`:

```
df1['employed'] = np.where(
    df1['emp_category'].eq('employed'),
    1,
    0
)
```

The variable `employed` is also called a *dummy variable*, a variable that can be thought of as an anwer to a Yes-or-No / True-or-False / 1-or-0 question.

The variable `employed` is now a numerical variable.

Let us treat our dataset from January 2015 as a *statistical population*.

Here our statistical population is concrete.

(b) How would you check that there is only one observation per individual in our selected dataset?

```
# your code goes here
```

There is only one entry per individual in our selected dataset. Therefore, we can refer to each individual by their identifiers:

```
statistical_population = (
    df1['person_id'].copy()
)
```

In the context of this thought experiment, let us compute the relative frequency of `employed==1` while ignoring missing values using `eq()` and `sum()` and let us name it `population_rel_freq`:

```
population_rel_freq = (
    # number of observations with employed==1
    df1['employed'].eq(1).sum()
    # divided by
    /
    # total number of observations
    df1['employed'].shape[0]
)
print(population_rel_freq)
```

(c) Compute the mean of the variable employed. How does it compare to the relative frequency of `employed==1`? Is it surprising?

```
population_mean = (
    # your code goes here
)
```

Let us randomly draw an individual from our example population and record whether the person is employed or not:

```
individual_drawn = rng.choice(
    a=statistical_population
)
random_draw = (
    df1
        .loc[df1['person_id'].eq(individual_drawn),
            ['person_id', 'employed']]
        .copy()
)
random_draw
```

This randomly drawn individual is classified as not employed (`employed==0`). This value can be called a *realisation* of the random variable *employed*.

Let us now draw a random sample of size `10`, i.e. 10 independent realisations of the random variable *employed*.

```
individuals_drawn = rng.choice(
        a=statistical_population,
        size=10
    )
random_sample = pd.DataFrame({
        'person_id': pd.Series(dtype=int),
        'employed': pd.Series(dtype=int)
    })
for i in range(10):
    individual_drawn = individuals_drawn[i]
    random_draw = (
        df1
            .loc[df1['person_id'].eq(individual_drawn),
                ['person_id', 'employed']]
            .reset_index(drop=True)
            .copy()
    )
    random_sample = pd.concat(
        [random_sample, random_draw],
        ignore_index=True
    )
random_sample
```

Since individuals are drawn with replacement (default behaviour of `choice()`), the same individual may be drawn more than once. The random draws are independent.

We can then compute the relative frequency of `employed=1` for this random sample:

```
(
    random_sample['employed'].eq(1).sum()
    /
    random_sample['employed'].shape[0]
)
```

(d) Compute the sample mean of the variable employed for this random sample. How does it compare to the sample relative frequency of `employed==1`?

```
sample_mean = (
    # your code goes here
)
```

Let us iteratively increase a random sample from size 1 to size `100000` by repeatedly carrying out random draws of an individual from the population, computing the sample size and the relative frequency of `employed==1` each time a new draw is added to the sample.

For efficiency, let us leverage the versatility of `.loc` (using `set_index()`) to avoid the use of `pd.concat()`.

Furthermore, let us use the `cumsum()` (cumulative sum) function/method when computing the mean of the current sample.

```
individuals_drawn = rng.choice(
    a=statistical_population,
    size=100000
)
random_samples = (
    df1
        .loc[:,['person_id', 'employed']]
        .set_index('person_id')
        .loc[individuals_drawn,:]
        .reset_index()
        .copy()
)

random_samples = (
    random_samples
        .reset_index(names='rank')
)
random_samples['size'] = (
    random_samples['rank'] + 1
)
random_samples['sum'] = (
    random_samples['employed'].cumsum()
)
random_samples['rel_freq'] = (
    random_samples['sum'] / random_samples['size']
)
```

With the help of this simulation, we can illustrate the convergence of sample relative frequencies to the population relative frequency.

```
fig, ax = plt.subplots()
ax.plot(
    random_samples['size'],
    random_samples['rel_freq'],
    label='Sample relative frequency')
ax.axhline(
    y=population_rel_freq,
    color='red',
    linestyle='--',
    label='Population relative frequency')

ax.set_xlabel('Sample size')
ax.set_ylabel('Relative frequency')
ax.legend()
```

Probabilities can be thought of as relative frequencies in the underlying statistical population. Another way to look at it is through a version of the *law of large numbers*: in repeated independent random sampling from the same population, as the sample size increases, sample relative frequencies converge to the corresponding probabilities in the population. This is related to the *frequentist* interpretation of probability.

The above figure provides an illustration for this version of the law of large numbers.

The argumentation is somewhat circular, but it is inevitable here, because randomness and probability are tightly related.

We can now answer the following question without problem:

(e) Suppose that an individual is drawn randomly from our statistical population. What is the probability that this individual is employed?

III. Conditional probabilities and the law of total probability

In Part II, we established the link between probabilities and relative frequencies. By analogy, we can extend this idea to conditional probabilities and conditional relative frequencies.

In real-world applications, we are often interested in conditional means, conditional probabilities, and/or conditional probability distributions. A typical basic **example of a conditional probability** is the probability that an individual becomes employed next month conditional on being unemployed this month:

$$\mathbf{P}[\text{employed next month} \mid \text{unemployed this month}]$$

Our `nlsy97_transitions.parquet` dataset contains information on employment status not only in the current month, but also in the following month, allowing us to analyse transition probabilities: besides the variable `emp_category`, there is a variable `emp_category_next`, which we can easily verify by running:

```
df.columns
```

How would you check that neither of the two variables (`emp_category` and `emp_category_next`) has missing values?

These two variables classify individuals by employment status into five categories: `employed`, `intermittently employed`, `unemployed`, `non-participant`, `military`.

Let us introduce a coarser classification of employment status, both for the current month and the next month, with only three categories. Let us name the new variables `status(t)` (for the current month) and `status(t+1)` (for the following month). Let us construct these variables using the replace() method, as shown below.

(a) Based on the code below, which categories are merged together?

```
replace_dictionary = {
    'intermittently employed': 'employed',
    'military': 'other',
    'non-participant': 'other',
}

df['status(t)'] = (
    df['emp_category']
        .replace(replace_dictionary)
)

df['status(t+1)'] = (
    df['emp_category_next']
        .replace(replace_dictionary)
)
```

Our first objective is to verify an important rule involving conditional probabilities, namely the *law of total probability*.

Let us introduce some notation first. Note that the notation is **arbitrary** and serves conciseness while providing flexibility.

Status abbreviations Let status *unemployed* be abbreviated as u. Let status *employed* be abbreviated as e. Let status *other* be abbreviated as o.

Unconditional probabilities Let $p(s(t))$ be the probability of having status s in month t:

$$p(s(t)) = \mathbf{P}[status(t) == s]$$

For example, $p(o(t+1))$ is the probability that a randomly selected individual has employment status *other* in month $t+1$.

Conditional probabilities Let $p(s(t+1)|c(t))$ be the probability of having status s in month $t+1$ conditional on having status c in month t.

$$p(s(t+1)|c(t)) = \mathbf{P}[status(t+1) == s \mid status(t) == c]$$

For example, $p(e(t)|u(t-1))$ is the probability of having status *employed* in month t conditional on having status *unemployed* in month $t-1$.

This notation allows flexibility. One can condition for example on destination status:

$$p(s(t)|c(t+1)) = \mathbf{P}[status(t) == s \mid status(t+1) == c]$$

For example, $p(o(t-1)|e(t))$ is the probability of having status *other* in month $t-1$ conditional on having status *employed* in month t.

According to the law of total probability, the probability of being unemployed in the following month can be written as:

$$p(u(t+1)) = \underbrace{p(u(t+1)|u(t))}_{\substack{\text{share of}\\ \text{unemployed}\\ \text{staying}\\ \text{unemployed}}} \cdot p(u(t)) + \underbrace{p(u(t+1)|e(t))}_{\substack{\text{share of}\\ \text{employed}\\ \text{becoming}\\ \text{unemployed}}} \cdot p(e(t)) + \underbrace{p(u(t+1)|o(t))}_{\substack{\text{share of}\\ \text{other}\\ \text{becoming}\\ \text{unemployed}}} \cdot p(o(t))$$

This is what we wish to check for every month in our data.

First, let us use the function `value_counts()` to compute the unconditional probabilities by status for the current month:

```
# the unconditional probabilities
# for the current month
current_shares = (
    df
        .groupby('year_month')['status(t)']
        .value_counts(normalize=True)
        .reset_index()
        .rename(columns={
            'proportion': 'p(status(t))'
        })
        .copy()
)
```

(b) Using the same approach, compute the shares for the following month (i.e. for `status(t+1)` instead of `status(t)`) and refer to the resulting data frame as `next_shares`:

```
# the unconditional probabilities
# for the following month
next_shares = (
    # your code goes here
)
```

Let us compute the conditional probabilities, i.e. the transition rates for each origin-status-destination-status pair in each month using the following code:

```
# the conditional probabilities
# of transitions across statuses
transition_rates = pd.crosstab(
    index=[df['year_month'], df['status(t)']],
    columns=df['status(t+1)'],
    normalize='index'
```

```
)
# naming the variables
transition_rates = (
    transition_rates
        .rename(columns = {
            c: 'p(' + c + '(t+1)|status(t))'
            for c in transition_rates.columns
        }
    )
)
transition_rates = (
    transition_rates.reset_index()
)
transition_rates.columns.name = None
```

Let us merge the data frame with the current shares (`current_shares`) and the data frame with the transition rates (`transition_rates`) together on `year_month` and `status(t)` using `pd.merge()`:

```
transitions = pd.merge(
    left=current_shares,
    right=transition_rates,
    how='inner',
    on=['year_month', 'status(t)']
)
```

(c) Subset / filter the `next_shares` data frame by keeping only the rows referring to the group of unemployed. Use `reset_index(drop=True)` after filtering. Refer to the new object as `comparison`.

```
comparison = (
    # your code goes here
)
```

(d) Applying the law of total probability:

Add a column named `p(unemployed(t+1),status(t))` (technical term: it is a *joint probability*) to the `transitions` data frame that is defined as the product of the current month shares (for each employment status) and the corresponding transition rates towards unemployment (conditional on employment status).

```
transitions['p(unemployed(t+1),status(t))'] = (
    # your code goes here
)
```

Let us now sum the joint probabilities across the three possible current-month employment statuses, separately for each month, using `groupby()` and then `agg()`. Let us refer to the resulting data frame as `to_be_compared`:

```
to_be_compared = (
    transitions
        .groupby('year_month')['p(unemployed(t+1),status(t))']
```

```
        .agg(**{'p(status(t+1)) formula': 'sum'})
        .reset_index()
        .copy()
)
```

(e) Using `pd.merge()` (already used between III.(b) and III.(c)), merge the `to_be_compared` data frame to the `comparison` data frame on `year_month`.

```
comparison = pd.merge(
    # your code goes here
)
```

Finally, we can check that, in all cases, the actual next-month probabilities are the same as computed using the law of total probability.

```
np.isclose(
    comparison['p(status(t+1))'],
    comparison['p(status(t+1)) formula']
).all()
```

(f) What does the result of `np.isclose(...).all()` tell us?

(g) What would the law of total probability look like for destination status `employed`? Put differently, how would you decompose $p(e(t))$ in terms of $p(e(t-1))$, $p(u(t-1))$, $p(o(t-1))$, $p(e(t)|e(t-1))$, $p(e(t)|u(t-1))$, $p(e(t)|o(t-1))$?